

Experiment No: 10

Experiment Title: Demonstration of Abstract Class and Interface in Java

Task 1: Shape Class with Abstract Method area()

Objectives:

1. To understand the concept of abstract classes in Java.
2. To learn how abstract methods are implemented in subclasses.
3. To practice inheritance and abstraction.
4. To calculate areas of different shapes.

Problem Analysis:

In this task, we create an abstract class named Shape containing an abstract method area(). Then we create two subclasses, Circle and Rectangle, which provide their own implementations of the area() method. This demonstrates abstraction by forcing subclasses to define the behavior of the abstract method.

Algorithm:

Step-1: Create an abstract class Shape.
Step-2: Declare an abstract method area().
Step-3: Create class Circle that extends Shape.
Step-4: Implement the area() method in Circle.
Step-5: Create class Rectangle that extends Shape.
Step-6: Implement the area() method in Rectangle.
Step-7: Create objects and call the area() method.

Source Code:

<pre>abstract class Shape { abstract void area(); } class Circle extends Shape { double radius = 7; void area() { double result = 3.1416 * radius * radius; System.out.println("Area of Circle = " + result); } } class Rectangle extends Shape { double length = 10; double width = 5;</pre>	<pre> void area() { double result = length * width; System.out.println("Area of Rectangle = " + result); } } public class ShapeMain { public static void main(String[] args) { Circle c = new Circle(); Rectangle r = new Rectangle(); c.area(); r.area(); } }</pre>
--	---

Output:

```
PS C:\Users\Laptop Click\OneDrive\Desktop\code\HasinJava> & 'C:\Program Files\Java\jdk-11.0.2\bin\java.exe' -cp 'C:\Users\Laptop Click\AppData\Roaming\Code\User\workspaceStorage\9488a1e7073b6f\bin' 'lab10.ShapeMain'
Area of Circle = 153.9384
Area of Rectangle = 50.0
PS C:\Users\Laptop Click\OneDrive\Desktop\code\HasinJava>
```

Task 2: ATMSERVICE Interface Implementation

Objectives:

1. To understand the concept of interfaces in Java.
2. To learn how a class implements an interface.
3. To practice abstraction using interfaces.
4. To perform basic ATM operations.

Problem Analysis:

In this task, we create an interface named ATMSERVICE containing methods withdraw(), deposit(), and checkBalance(). Then we create a class DBBL that implements the interface and provides definitions for all methods. This demonstrates how interfaces achieve abstraction in Java.

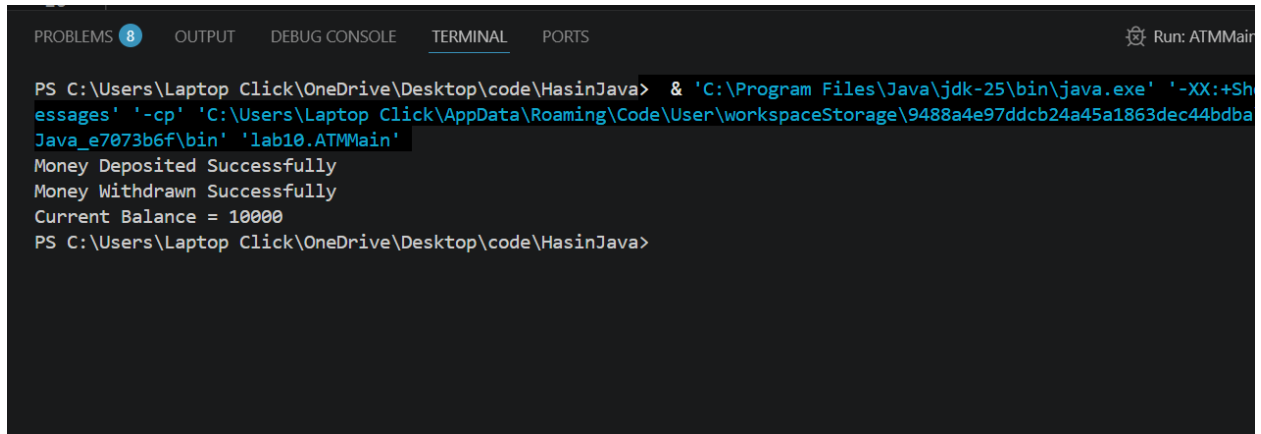
Algorithm:

- Step-1: Create an interface ATMSERVICE.
- Step-2: Declare methods withdraw(), deposit(), and checkBalance().
- Step-3: Create class DBBL that implements ATMSERVICE.
- Step-4: Define all interface methods in DBBL.
- Step-5: Create an object of DBBL.
- Step-6: Call all methods.
- Step-7: Display ATM operation messages.

Source Code:

<pre>interface ATMSERVICE { void withdraw(); void deposit(); void checkBalance(); } class DBBL implements ATMSERVICE { public void withdraw() { System.out.println("Money Withdrawn Successfully"); } public void deposit() { System.out.println("Money Deposited Successfully"); } public void checkBalance() { System.out.println("Current Balance = 10000"); } } public class ATMMain { public static void main(String[] args) { DBBL account = new DBBL();</pre>	<pre>account.deposit(); account.withdraw(); account.checkBalance(); } }</pre>
---	--

Output:

A screenshot of a Java IDE's terminal window. The terminal shows the execution of a Java program. The command prompt is 'PS C:\Users\Laptop Click\OneDrive\Desktop\code\HasinJava>'. The command executed is '& 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+Sh...essages' '-cp' 'C:\Users\Laptop Click\AppData\Roaming\Code\User\workspaceStorage\9488a4e97ddcb24a45a1863dec44bdba\Java_e7073b6f\bin' 'lab10.ATMMain''. The output of the program is: 'Money Deposited Successfully', 'Money Withdrawn Successfully', and 'Current Balance = 10000'. The terminal window has tabs for 'PROBLEMS 8', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The top right corner shows a 'Run: ATMMain' button.

```
PS C:\Users\Laptop Click\OneDrive\Desktop\code\HasinJava> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+Sh...essages' '-cp' 'C:\Users\Laptop Click\AppData\Roaming\Code\User\workspaceStorage\9488a4e97ddcb24a45a1863dec44bdba\Java_e7073b6f\bin' 'lab10.ATMMain'
Money Deposited Successfully
Money Withdrawn Successfully
Current Balance = 10000
PS C:\Users\Laptop Click\OneDrive\Desktop\code\HasinJava>
```

Discussion & Conclusion:

This experiment helped us understand two important concepts of Object-Oriented Programming: abstraction and interfaces. In the first task, an abstract class Shape was used to define a common method area(), which was implemented differently by Circle and Rectangle. In the second task, the ATMService interface defined ATM operations, and the DBBL class implemented those operations. These examples show how abstraction helps hide implementation details while providing a clear structure for programs. Overall, abstract classes and interfaces make Java programs more flexible, organized, and reusable.